T06 Enemy AI

CRE132 worksheet

What you'll make

Slimes with a simple "brain". They sit, then hop about near where they started. Walk close and a slime chases you; run far enough away and it gives up, sits, and goes back to wandering near home. It can't hurt you yet: T06b adds the attack.

The brain is a **state machine**: the slime is always in exactly one **state** (Idle, Wander or Chase), and each state has its own short rules for what to do and when to switch. It's the longest script so far, so you'll build it a piece at a time.



Figure 1: The finished meadow: the player on a patch of dirt, the edge of the pond at the top and a row of bushes below.

Before you start

1. Unzip `CRE132_T06_EnemyAI_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_T06_EnemyAI_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/TopDown/T06_EnemyAI`.
3. Press **Play**. You can walk and swing the sword (J), and the two slimes can be hit, but they don't move. Press Play again to stop.

Under **Characters** are the **Player** (T04's, plus **Health** 10 and **HealthTint**) and **Slime 1** and **Slime 2**. Each slime already has a **Rigidbody 2D** (**Gravity Scale** 0, rotation frozen), a **Box Collider 2D** at its feet, **Health** (3), **HealthTint**, and a **Sprite** child whose Animator uses the `SlimeAI` controller. **Slime 2** lives behind the pond.

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. The idea: three states

Before any code, draw this on paper: three boxes, with an arrow for each rule.

State	What it does	When it leaves
Idle	waits a random time	player within 4 units: **Chase**; time's up: **Wander**
Wander	walks to a random spot near home	player within 4: **Chase**; got there (or gave up): **Idle**
Chase	runs at the player	player beyond 7: **Idle**

A state machine breaks one big question ("what should the enemy do?") into small ones: "what do I do in *this* state, and when do I leave it?" Only one state runs at a time, so their rules can't fight each other.

Open the **Animator** window with a slime's **Sprite** selected: `SlimeAI` has states such as **Slime_Idle** and **Slime_Move**, with no arrows between them. The script picks one itself with `animator.Play`.

2. Starting the script: the states and the settings

Make a folder `Assets/_game/Scripts/T06_EnemyAI` and create `EnemyAI` in it. Open it. Unity has already written the class line, `public class EnemyAI : MonoBehaviour`, with an opening `{` below it and a closing `}` at the very end. Keep those three lines. Delete the `Start` and `Update` methods between the braces, and replace the `using UnityEngine;` line at the top with:

```
// A simple enemy "brain" built as a state machine. The enemy is always in exactly one
// state, and each state has its own rules for what to do and when to switch to another
// state:
//   Idle       wait a moment, then Wander (or Chase if the player comes close)
//   Wander     walk to a random spot near home, then Idle (or Chase if the player comes
//              close)
//   Chase      run at the player; back to Idle if the player gets away
// T06b adds two more states: Attack (jump on the player) and ReturnHome (walk back before
// wandering). It moves in straight lines, so trees and water get in its way: T07 fixes
// that with pathfinding.
using UnityEngine;
```

Everything else goes **inside the class's braces**, in the order given. Unity will show red errors in the Console until every part is in: that's expected, so don't worry about them until part 7. Start with the list of states and the settings:

```
    // Every state the enemy can be in. An enum is a list of named options.
    private enum State
    {
        Idle,
        Wander,
        Chase
    }

    [Header("Idle and wander")]
    [SerializeField] private float wanderSpeed = 1f;
```

```
    // How far from home it wanders.
    [SerializeField] private float wanderRadius = 3f;
    [SerializeField] private float minIdleTime = 1f;
    [SerializeField] private float maxIdleTime = 3f;
    // Gives up on a spot it can't reach (a tree in the way) after this long.
    [SerializeField] private float maxWanderTime = 4f;

    [Header("Chase")]
    [SerializeField] private float chaseSpeed = 2.5f;
    // Starts chasing when the player is this close.
    [SerializeField] private float detectionRange = 4f;
    // Gives up when the player is this far away. It's bigger than detectionRange, so the
    // enemy doesn't flick between chasing and giving up when the player stands right on
    // the edge.
    [SerializeField] private float giveUpRange = 7f;

    // Shown in the Inspector so you can watch it change during Play.
    [SerializeField] private State currentState = State.Idle;

    private Rigidbody2D body;
    private Animator animator;
    private Transform player;
    private Vector2 homePosition;
    private Vector2 wanderTarget;

    // Set by the states each frame; FixedUpdate turns them into a velocity.
    private Vector2 moveDirection;
    private float moveSpeed;

    // Seconds spent in the current state. ChangeState sets it back to 0.
    private float stateTimer;
    private float idleTime;
```

- **An `enum` is a list of named options.** `State.Chase` is easier to read than "state number 2", and C# won't let you type a state that doesn't exist.
- **`currentState` has `[SerializeField]`**, so it shows in the Inspector and you can watch it change during Play.
- **`[Header("...")]`** puts a bold title above a group of settings in the Inspector.
- **`detectionRange` (4) and `giveUpRange` (7) are different on purpose.** Part 9 shows why.

3. Getting ready and running one state

Below the fields, add:

```
    private void Awake()
    {
        body = GetComponent<Rigidbody2D>();
        // The Animator is on the Sprite child (see T03).
        animator = GetComponentInChildren<Animator>();
    }

    private void Start()
    {
        homePosition = transform.position;

        GameObject playerObject = GameObject.FindWithTag("Player");
        if (playerObject != null)
        {
```

```
            player = playerObject.transform;
        }

        ChangeState(State.Idle);
    }

    private void Update()
    {
        stateTimer = stateTimer + Time.deltaTime;

        // Run the rules for the current state only.
        switch (currentState)
        {
            case State.Idle:
                UpdateIdle();
                break;
            case State.Wander:
                UpdateWander();
                break;
            case State.Chase:
                UpdateChase();
                break;
        }
    }

    private void FixedUpdate()
    {
        body.linearVelocity = moveDirection * moveSpeed;
    }
```

- **`Start` remembers home** (where the slime starts) and finds the player by its **Player** tag.
- **`Update` uses a `switch`**: it looks at `currentState` and runs only that state's method. Each method does its job and decides whether to change state.
- **The states never touch the Rigidbody.** They only set `moveDirection` and `moveSpeed`, and `FixedUpdate` turns those into a velocity: the same split as T01's player.

4. ChangeState: the one place states change

Below `FixedUpdate`, add:

```
    // Switches state. Everything that should happen once, as a state starts, goes here.
    private void ChangeState(State newState)
    {
        Debug.Log(gameObject.name + ": " + currentState + " -> " + newState);
        currentState = newState;
        stateTimer = 0f;
        moveDirection = Vector2.zero;

        switch (newState)
        {
            case State.Idle:
                // A random wait, so several enemies don't all move at the same moment.
                idleTime = Random.Range(minIdleTime, maxIdleTime);
                PlayAnimation("Slime_Idle");
                break;
            case State.Wander:
                // A random spot inside a circle around home, so it never wanders off for
                // good.
```

```
                wanderTarget = homePosition + Random.insideUnitCircle * wanderRadius;
                moveSpeed = wanderSpeed;
                PlayAnimation("Slime_Move");
                break;
            case State.Chase:
                moveSpeed = chaseSpeed;
                PlayAnimation("Slime_Move");
                break;
        }
    }
```

- **Every change of state goes through this method.** That gives one place to log the change (the `Debug.Log` line) and one place for each state's "on start" jobs.
- **It resets `stateTimer` and stops moving**, so every state starts fresh.
- **The second `switch` does the start-of-state jobs**: pick an idle time, pick a wander spot, set the speed, play the right animation.
- **`Random.insideUnitCircle * wanderRadius`** is a random point inside a circle of that radius. Added to `homePosition`, the slime never wanders far from home.

5. Idle and Wander

Below `ChangeState`, add:

```
    private void UpdateIdle()
    {
        if (PlayerIsClose())
        {
            ChangeState(State.Chase);
        }
        else if (stateTimer >= idleTime)
        {
            ChangeState(State.Wander);
        }
    }

    private void UpdateWander()
    {
        if (PlayerIsClose())
        {
            ChangeState(State.Chase);
        }
        else if (HasReached(wanderTarget) || stateTimer >= maxWanderTime)
        {
            ChangeState(State.Idle);
        }
        else
        {
            MoveTowards(wanderTarget);
        }
    }
```

- **Both check for the player first**, so a slime notices you whatever it was doing.
- **Wander gives up after `maxWanderTime`** (4 seconds). If a tree is in the way, the slime would otherwise push against it for ever.

6. Chase

```
    private void UpdateChase()
    {
        // The player may have been destroyed (died).
        if (player == null)
        {
            ChangeState(State.Idle);
            return;
        }

        float distance = Vector2.Distance(transform.position, player.position);
        if (distance > giveUpRange)
        {
            ChangeState(State.Idle);
        }
        else
        {
            MoveTowards(player.position);
        }
    }
```

- **`player == null`** happens if the player has been destroyed. Without this check, the next line would throw a NullReferenceException.
- **Too far: give up and go back to Idle. Otherwise: keep running at the player.**

7. The helpers and the gizmos

Add the helper methods that the states share, and finally the gizmos. The class's closing `}` comes after all of this:

```
    // ---------- Helpers the states share ----------

    private bool PlayerIsClose()
    {
        if (player == null)
        {
            return false;
        }
        return Vector2.Distance(transform.position, player.position) <= detectionRange;
    }

    private void MoveTowards(Vector2 target)
    {
        Vector2 toTarget = target - (Vector2)transform.position;
        moveDirection = toTarget.normalized;
    }

    private bool HasReached(Vector2 target)
    {
        return Vector2.Distance(transform.position, target) < 0.1f;
    }

    private void PlayAnimation(string stateName)
    {
        if (animator != null)
        {
            animator.Play(stateName);
        }
```

```
    }

    // Draws the ranges in the Scene view when the enemy is selected.
    private void OnDrawGizmosSelected()
    {
        Vector3 home = transform.position;
        if (Application.isPlaying)
        {
            home = homePosition;
        }

        Gizmos.color = Color.green;
        Gizmos.DrawWireSphere(home, wanderRadius);
        Gizmos.color = Color.yellow;
        Gizmos.DrawWireSphere(transform.position, detectionRange);
        Gizmos.color = Color.grey;
        Gizmos.DrawWireSphere(transform.position, giveUpRange);
    }
```

- **`MoveTowards`** only sets a direction: subtract to get from here to the target, then normalise, as in T05.
- **`HasReached`** says "close enough" at 0.1 units. Checking for exactly the same position would almost never be true.
- **`PlayAnimation`** checks the Animator exists first, so a slime without one still works.
- **`OnDrawGizmosSelected`** draws the ranges in the Scene view when a slime is selected.

Save. The Console should now be clear of red errors. If it isn't, the first error's line number tells you where to look; compare that part with the listing.

Add it: select **Slime 1** and **Slime 2** (the roots, not their Sprites) and add **Enemy AI** to both. Leave every value as it is: **Wander Speed** 1, **Wander Radius** 3, **Min Idle Time** 1, **Max Idle Time** 3, **Max Wander Time** 4, **Chase Speed** 2.5, **Detection Range** 4, **Give Up Range** 7.

Test it: press Play. The slimes wait, then hop about. Walk close to **Slime 1**: it chases you. Run away and it gives up and sits down. Hit it three times with J to remove it.

8. Watching the brain work

- **Inspector:** select a slime during Play and watch **Current State** change.
- **Console:** every change is logged, for example `Slime 1: Wander -> Chase`.
- **Gizmos:** select a slime in the Scene view to see the wander circle (green), detection range (yellow) and give-up range (grey).

9. Why two ranges?

Stop Play, select **Slime 1** and set **Give Up Range** to **4**, the same as **Detection Range**. Press Play and stand right on the edge of the yellow circle. The slime flickers between **Chase** and **Idle**, and the Console fills up. With two different ranges there's a gap where neither rule fires, so it settles. Set **Give Up Range** back to **7**.

10. Something to notice

Walk to the bottom of the pond so that **Slime 2** is on the other side, and let it notice you. It runs straight at you, into the water's edge, and gets stuck. It only knows "move towards", not how to get round things. T07 fixes that with pathfinding.

Check your work

- ☐ The slimes wait, then wander near where they started.

- ☐ Walking within 4 units makes a slime chase you.
- ☐ Running more than 7 units away makes it give up and go back to idling and wandering.
- ☐ **Current State** in the Inspector and the Console show each change.
- ☐ Three sword hits remove a slime.
- ☐ Compare with `CRE132_T06_EnemyAI_Finished` if something behaves differently.

Key points

- **A state machine** breaks a big "what should the enemy do?" into small questions: "what do I do in *this* state, and when do I leave it?"
- **Only one state runs at a time**, so its rules can't fight each other.
- **Changing state in one method** (`ChangeState`) gives one place for start-of-state jobs, and one place to log or debug.
- **Two different ranges** stop flickering at a boundary.

If something goes wrong

- **Lots of red errors while typing:** normal until all of part 7 is in. If they stay, check every method is inside the class's braces and that you didn't delete the final `}`.
- **"The name 'PlayAnimation' does not exist" (or another helper):** part 7 is missing or is outside the class.
- **The slimes never chase:** the **Player** must have the **Player** tag (`FindWithTag` finds it by that).
- **The slime slides about without animating:** the animation names in `PlayAnimation("...")` must match the `SlimeAI` states exactly: **Slime_Idle**, **Slime_Move**.
- **The slime drifts down the screen or spins:** its **Rigidbody 2D** needs **Gravity Scale** 0 and **Freeze Rotation Z**.

Try it

1. Make the slime notice the player only if it can see them: a `Physics2D.Linecast` that isn't blocked by trees.
2. Add a leash: give up when the slime is too far from **home**, not just from the player.